针对“端到端加密即时通讯软件”的课程设计目标，建议将原来的时间表改为“按完整实现流程推进”的开发规划。这样更适合实际落地，也更方便后续直接照着文档逐项实现、测试和汇报。

本次重规划的核心原则是：

1. 去掉按周按天的排期，只保留完整开发流程与优先级。
2. 不再采用过重的多栈混合方案，而是尽量基于当前仓库已有的 Python 基础演进。
3. 在保证课程要求的前提下，优化成品效果，使界面、演示和安全验证都更完整，同时控制实现复杂度。

一、推荐技术路线（兼顾效果与可实现性）

结合当前项目中已经存在的 RSA 核心代码与桌面 GUI 原型，更推荐采用下面这条路线，而不是直接上 React + Electron + Node.js 的重型组合：

- 服务端：Python FastAPI + WebSocket
  - 负责用户连接、消息转发、在线状态维护。
  - 服务端只做盲转发，不保存明文消息，不保存任何客户端私钥。
- 桌面端：基于当前 Python GUI 继续演进
  - 可保留 Tkinter 架构，并通过 ttkbootstrap 或 CustomTkinter 提升界面观感。
  - 打包为 Windows 可执行程序，便于课程展示。
- Web 端：原生 HTML/CSS/JavaScript + WebSocket
  - 只做轻量版浏览器客户端，重点支持文本聊天、密钥交换展示和消息接收发送。
  - 避免前端框架带来的额外工程负担。
- 密码学方案：RSA + AES-GCM 混合加密
  - RSA：用于公钥交换、AES 会话密钥加密、身份标识。
  - AES-GCM：用于实际聊天消息和文件内容加密。
  - 该方案比“所有内容直接用 RSA 分块加密”更快、更合理，也更符合真实通信系统设计。
- 桌面端打包：PyInstaller
  - 直接输出 Windows .exe，适合作为最终课程交付物。

二、项目最终成品目标

最终成品建议明确收敛为以下可演示、可验收的目标：

- 一个云端 WebSocket 中继服务器。
- 一个 Windows 桌面聊天客户端。
- 一个可在浏览器打开的轻量 Web 聊天客户端。
- 支持桌面端和桌面端之间的安全聊天。
- 支持桌面端和 Web 端之间的跨端安全聊天。
- 支持文本消息的端到端加密传输。
- 支持小文件加密传输。
- 支持在界面中可视化展示“明文 -> 密钥 -> 密文 -> 解密还原”的过程。
- 支持 Wireshark 抓包验证公网传输内容为密文。
- 支持最终导出为带注释源码、报告、PPT、演示程序的完整交付包。

三、项目范围控制建议

为了保证项目真正能完成，建议明确“必做项”和“非必须项”。

必做项

- 一对一即时聊天。
- RSA 密钥对生成、导入、导出。
- AES 会话密钥生成与交换。
- 文本消息端到端加密。
- 文件端到端加密传输。
- 桌面端与 Web 端互通。
- 加解密过程可视化。
- 云服务器部署。
- Wireshark 抓包验证。

建议暂不做的功能

- 群聊。
- 音视频通话。
- 复杂用户注册登录系统。
- 消息长期云端存储。
- 朋友圈、头像上传、历史搜索等非核心社交功能。

说明：只做一对一聊天，能明显降低协议复杂度，但不影响课程设计对算法、密钥交换、网络通信和界面展示的要求。

四、推荐的系统结构

建议把整个项目整理为如下结构，后续实现和汇报都更清晰：

```
InfoSecurityProj/
├─ server/
│  ├─ app.py
│  ├─ ws_handler.py
│  └─ protocol.py
├─ client_desktop/
│  ├─ app.py
│  ├─ ui/
│  ├─ network/
│  └─ storage/
├─ web_client/
│  ├─ index.html
│  ├─ app.js
│  └─ style.css
├─ crypto/
│  ├─ rsa_core.py
│  ├─ aes_core.py
│  └─ message_crypto.py
├─ docs/
│  ├─ architecture.md
│  ├─ protocol.md
│  └─ screenshots/
├─ Demonstration/
└─ Project_Plan_and_Requirements.md
```

如果不想一开始就调整目录，也可以先在当前结构下开发，最后再统一整理。

五、完整开发流程（无时间轴版本）

以下流程建议按照“先打通主链路，再做效果优化”的顺序推进。

1. 先确定通信模型与消息协议

- ☐ 明确系统角色：客户端、服务端、中继通道。
- ☐ 明确服务端职责：只负责连接管理和消息转发，不参与解密。
- ☐ 定义客户端唯一标识，例如用户名、设备 ID 或自定义昵称。
- ☐ 设计统一消息格式，至少包括：
  - ☐ 消息类型 type
  - ☐ 发送方 sender_id
  - ☐ 接收方 receiver_id
  - ☐ 时间戳 timestamp
  - ☐ 会话编号 session_id
  - ☐ 密文 ciphertext
  - ☐ 随机数 nonce
  - ☐ 附加认证数据或标签 tag
  - ☐ 必要时携带的加密 AES 密钥 wrapped_key
- ☐ 统一错误消息和回执消息格式，方便调试和界面提示。

建议的消息封装格式如下：

```
{
    "type": "chat_message",
    "sender_id": "client_a",
    "receiver_id": "client_b",
    "session_id": "session_001",
    "timestamp": "2026-04-15T12:00:00Z",
    "nonce": "base64_nonce",
    "ciphertext": "base64_ciphertext",
    "tag": "base64_tag",
    "wrapped_key": "base64_wrapped_aes_key"
}
```

2. 对现有密码学模块进行改造

当前仓库已有 RSA 功能，可以继续复用，但需要从“单机 RSA 加解密工具”升级为“即时通信加密模块”。

- ☐ 保留 RSA 密钥对生成、导入、导出功能。
- ☐ 保留 RSA-OAEP 作为 AES 会话密钥加密方案。
- ☐ 新增 AES-GCM 模块，用于消息正文和文件内容加密。
- ☐ 将“RSA 直接加密整段消息”的思路改为“RSA 加密 AES 密钥，AES 加密真实消息内容”。

- ☐ 为每次会话或每次文件传输生成新的随机 AES 密钥。
- ☐ 输出密钥指纹（例如 SHA-256 指纹）用于双方人工核验公钥。
- ☐ 为消息解密失败、密钥不匹配、数据篡改等情况定义明确异常提示。

3. 设计并实现密钥交换流程

建议采用下面这套更合理的握手逻辑：

1. 客户端启动时生成或加载本地 RSA 公私钥对。
2. 客户端连接服务器后上报自己的用户 ID 与公钥。
3. 发起聊天的一方获取对方公钥。
4. 发送方生成随机 AES 会话密钥。
5. 发送方使用对方 RSA 公钥加密 AES 会话密钥。
6. 发送方使用 AES-GCM 加密实际聊天内容。
7. 发送方将“RSA 加密后的 AES 密钥 + AES 密文 + nonce + tag”一起发给对方。
8. 接收方使用自己的 RSA 私钥解密得到 AES 会话密钥。
9. 接收方再使用 AES-GCM 解密还原明文。

可选增强：

- ☐ 会话建立后缓存本次 AES 会话密钥，减少频繁握手。
- ☐ 设定会话轮换机制，例如切换联系人时重新生成新密钥。
- ☐ 文件传输单独使用一次性 AES 密钥，避免和文本消息共用。

4. 先打通服务端中继主链路

- ☐ 基于 FastAPI 或 websockets 库实现 WebSocket 服务器。
- ☐ 实现客户端注册上线功能。
- ☐ 实现在线用户列表广播或查询接口。
- ☐ 实现按接收方 ID 路由转发消息。
- ☐ 实现公钥上传、查询与转发。
- ☐ 实现心跳机制，避免断线状态不准确。
- ☐ 服务端日志只记录连接信息、消息类型、消息长度，不记录任何明文。
- ☐ 服务端不落盘保存敏感通信内容。

建议服务端只维护以下最小信息：

- 在线用户 ID
- 当前 WebSocket 连接
- 用户公钥或公钥指纹

- 最近连接时间

5. 开发桌面端主客户端

桌面端建议作为主展示端，因为当前仓库已经有 GUI 基础，最容易做出完整效果。

- ☐ 在现有界面基础上升级为聊天式布局。
- ☐ 左侧区域显示联系人列表、在线状态、连接状态。
- ☐ 中间区域显示消息气泡、输入框、发送按钮、文件发送按钮。
- ☐ 右侧区域显示加密控制台或安全状态面板。
- ☐ 支持本地生成密钥、加载密钥、导出公钥。
- ☐ 支持选择联系人并发起加密会话。
- ☐ 支持发送文本消息。
- ☐ 支持发送文件并显示传输状态。
- ☐ 支持显示消息发送中、已送达、已解密等状态。
- ☐ 支持断线重连与失败提示。

桌面端建议重点突出以下展示元素：

- 安全通道已建立
- 对方公钥指纹
- 当前加密算法
- 本次消息使用的 AES 会话编号
- 原文长度 / 密文长度对比

6. 开发轻量 Web 客户端

Web 端不建议追求过度复杂，而应该聚焦“跨平台演示”这一核心目标。

- ☐ 页面支持输入昵称或设备 ID。
- ☐ 页面支持连接到云服务器。
- ☐ 页面支持生成或导入本地公钥信息。
- ☐ 页面支持选择联系人并发送文本消息。
- ☐ 页面支持展示收到的密文和解密结果。
- ☐ 页面支持展示简化版加密控制台。
- ☐ 页面适配手机和平板浏览器宽度。

建议策略：

- 桌面端做完整版功能。
- Web 端做轻量版功能。

这样既满足跨端要求，也不会把工作量拉得过高。

7. 做出课程汇报最关键的可视化亮点

“可视化加解密流程”是整个项目最容易打动老师的部分，必须单独设计。

- ☐ 在界面右侧设计“Crypto Console”面板。
- ☐ 将一次消息发送流程拆成多条可视化日志。
- ☐ 区分发送方视角和接收方视角。
- ☐ 对不同阶段使用不同颜色或图标标识。

推荐展示内容如下：

- [原文] 你好
- [生成 AES 会话密钥] 8F A1 ...
- [使用对方 RSA 公钥加密会话密钥] 成功
- [生成密文] U2FsdGVk...
- [发送到服务器] 成功
- [服务器盲转发] 已转发
- [接收方收到密文] U2FsdGVk...
- [使用本地 RSA 私钥解密会话密钥] 成功
- [AES-GCM 解密还原] 你好

这部分既能体现算法流程，也能直观说明“为什么抓包看到的是密文”。

8. 文件传输功能按“小而完整”的方式实现

文件传输建议控制范围，不追求大文件和断点续传，而是实现一个稳定可演示的版本。

- ☐ 限制单次文件大小，例如 5MB 或 10MB。
- ☐ 文件内容使用 AES-GCM 加密。
- ☐ 文件 AES 密钥使用接收方 RSA 公钥加密。
- ☐ 文件消息中携带文件名、大小、类型等必要元数据。
- ☐ 接收方完成解密后可另存为原始文件。
- ☐ 对不支持的格式或过大文件给出明确提示。

9. 联调与异常场景处理

在功能完成后，不要只测“正常情况”，还要补足异常分支。

- ☐ 测试公钥未导入时发送消息的提示。

- ☐ 测试私钥错误时解密失败的提示。
- ☐ 测试消息被篡改时 AES-GCM 校验失败的提示。
- ☐ 测试对方离线时的发送失败提示。
- ☐ 测试网络中断后的重连逻辑。
- ☐ 测试桌面端与桌面端互通。
- ☐ 测试桌面端与 Web 端互通。

10. 完成安全验证与演示证明

Wireshark 抓包验证必须纳入正式流程，而不是最后临时补。

- ☐ 在局域网或公网环境部署服务器。
- ☐ 使用过滤条件锁定目标通信端口。
- ☐ 在聊天时抓取 WebSocket 或 TCP 载荷。
- ☐ 截图保留密文传输画面。
- ☐ 对照客户端界面中的明文内容，证明传输链路中不可直接看到明文。
- ☐ 在报告中明确说明：服务端无法直接恢复聊天内容，因为服务端没有客户端私钥。

11. 整理文档、注释与最终交付物

- ☐ 为关键代码补充注释，重点说明密钥交换、消息加解密、网络收发流程。
- ☐ 绘制系统架构图。
- ☐ 绘制密钥交换流程图。
- ☐ 绘制消息发送与解密时序图。
- ☐ 截取桌面端界面图、Web 端界面图、抓包图、可视化控制台图。
- ☐ 撰写程序使用说明。
- ☐ 撰写测试章节，包含正常测试与异常测试。
- ☐ 打包 Windows 可执行程序。
- ☐ 整理报告、源码、截图、PPT 与演示视频素材。

六、建议的功能优先级

为了避免开发过程中越做越散，建议按照下面优先级推进。

第一优先级：先做通主链路

- 本地 RSA 密钥管理

- AES-GCM 消息加密
- 服务端消息中继
- 桌面端一对一文本聊天
- 加密消息成功收发

第二优先级：做跨端与可视化

- Web 端文本聊天
- 桌面端与 Web 端互通
- 加解密控制台
- 在线状态与送达状态显示

第三优先级：做成品包装与加分项

- 文件传输
- 公钥指纹校验
- Windows 打包
- Wireshark 抓包展示
- PPT 与报告整理

七、建议保留的课程亮点

为了让最终展示更有说服力，建议把亮点集中在以下几个方面：

1. 不是普通聊天室，而是端到端加密聊天室。
2. 服务端只转发密文，无法查看聊天明文。
3. 桌面端和 Web 端可以互通，体现跨端能力。
4. 右侧加密控制台可以实时展示算法执行流程。
5. Wireshark 抓包可以直观看到传输内容为密文。
6. 使用 RSA + AES-GCM 的混合加密设计，技术路线更规范。

八、最终验收标准

当以下条件全部满足时，就说明项目已经达到较完整的课程设计交付标准：

- ☐ 两个客户端可以通过云服务器建立通信。
- ☐ 文本消息在公网传输中为密文。
- ☐ 接收端可以正确解密还原明文。
- ☐ 服务端无法直接读取消息内容。
- ☐ 桌面端运行稳定，可打包为 exe。
- ☐ Web 端可在浏览器中访问并完成基础聊天。
- ☐ 界面中能直观看到加密与解密过程。
- ☐ 有完整测试记录、抓包截图、系统截图和说明文档。

九、结论

新的规划不再强调“几天做什么”，而是强调“先做什么、后做什么、哪些必须做、哪些可以裁剪”。

如果以当前仓库为基础推进，最合理的路线是：

- 保留并改造现有 RSA 模块。
- 新增 AES-GCM 消息加密模块。
- 使用 Python WebSocket 服务端做盲转发。
- 以桌面端为主展示端，Web 端为跨端补充端。
- 把可视化加解密流程和 Wireshark 抓包证明作为最终展示重点。

这样改完之后，整个计划会更聚焦、更能实现，最终成品在“技术深度、界面表现、演示效果、课程答辩说服力”这几个方面也会更均衡。